
CSE321: Operating Systems

Theory Assignment 01

*A Comparative Analysis of
Multi-Threading and Event-Driven Concurrency*

Submitted by: Aksan Gony Alif

Student ID: 24341256

Section: 25

Course: Operating Systems

Date: October 9, 2026

Contents

1	Introduction	2
2	Thread-Based Concurrency	2
2.1	Handling Concurrent Tasks with Multiple Threads	2
2.2	Thread Scheduling and Context Switching	2
2.3	Synchronization Mechanisms	3
2.4	Advantages and Limitations	4
3	Event Loop Architecture	4
3.1	Single-Threaded Event Loop Operation	4
3.2	Callbacks, Event Queues, and Non-Blocking Operations	5
3.3	Callbacks, Promises, and async/await	5
3.4	Strengths and Challenges	6
4	Asynchronous I/O Systems and Performance Analysis	7
4.1	Synchronous versus Asynchronous I/O	7
4.2	I/O-Bound versus CPU-Bound Workloads	7
4.3	Performance Under High Concurrency	8
4.4	Real-World Use Cases	8
5	Conclusion	9

1. Introduction

Modern software systems are expected to serve thousands of concurrent users while remaining responsive, fault tolerant, and resource efficient. To achieve this, two dominant concurrency paradigms have emerged. The first one is **thread-based concurrency**, which has been the mainstay of operating systems since the 1990s. And the second one is **event-driven concurrency**, popularized by runtime environments such as Node.js, Nginx, and Python's `asyncio`. Each model has a completely different way of doing a lot of various tasks at an instant moment. Threads depend on the operating system's scheduler to switch up different execution contexts on the CPU cores that are available. Event-driven systems, on the other hand, use a single-threaded event loop to handle a queue of small, non-blocking work items.

This report analyses the architectural foundations, trade-offs, and real-world applicability of both models.

2. Thread-Based Concurrency

2.1 Handling Concurrent Tasks with Multiple Threads

A *thread* is the smallest unit of execution that can be independently scheduled by the operating system. In a single process, multiple threads share the same address space, file descriptors, and global variables. However, each thread has its own program counter, registers, and stack. Traditional multi-threaded systems exploit this structure by spawning one thread per concurrent task, for example one thread per incoming network connection in a classical Apache-style web server.

Because all threads in a process share memory, communication between them is fast and direct without having the cost of expensive IPC methods. A producer thread can simply write to a shared data structure that a consumer thread reads. This shared memory model makes parallel decomposition natural. A computational task can be split into chunks, each handed to a thread, and the results gathered when all threads complete using `join()`. On a multi-core processor, the operating system can dispatch threads to different cores and ensure genuine *parallelism* as opposed to mere logical concurrency.

The programmer has to think carefully about the order in which threads access shared data because it makes things easier in multi core programming. Two threads writing to the same variable without coordinating can lead to results that depend on the exact order in which their instructions are run, which is a common source of bugs that are hard to reproduce.

2.2 Thread Scheduling and Context Switching

The operating system's scheduler decides which thread runs on which CPU core at any given moment. Most modern schedulers (such as Linux's Completely Fair Scheduler, CFS) use a preemptive, priority-based policy where each thread is granted a short time slice. Here, a timer interruption allows the kernel to suspend it and dispatch another runnable thread. This is what creates the illusion that many threads are executing simultaneously even on a single core. But when we have multiple cores to run multiple threads at the same time, it creates both concurrency in the perspective of any single core executing and parallelism in the perspective of

multiple cores running in a single moment.

The mechanism that makes concurrency possible in a single core CPU is the **context switch**. When the scheduler decides to switch processes, the kernel saves the current process's CPU registers, program counter, and stack pointer into its process control block (PCB), then loads the corresponding state of the next thread. If the two threads belong to different processes, the page table base register is also updated and the translation lookaside buffer (TLB) may need to be flushed. A context switch typically costs between one and ten microseconds on modern hardware. But it indirectly costs cache pollution, TLB misses, and pipeline stalls. When thousands of threads compete for a small number of cores, these costs add up and become a dominant factor in overall performance.

2.3 Synchronization Mechanisms

The operating system and standard libraries give threads synchronization primitives so that they can all use shared resources at the same time.

A **mutex** (mutual exclusion lock) makes sure that only one thread can run a critical section at a time. To enter the section, a thread must get the mutex. If another thread already has it, the requesting thread is blocked until the mutex is released. Atomic compare-and-swap or test-and-set instructions are often used to make mutexes. When there is contention, the kernel sets the thread that is waiting to sleep.

A **semaphore** is like a mutex, but it uses an integer counter instead of a binary lock. A counting semaphore that starts at N lets up to N threads run at the same time. This is useful for limiting how many threads can access a pool of resources, like database connections, at the same time. You can also use binary semaphores to send signals between threads. In this case, one thread waits for an event that another thread will eventually signal.

Condition variables work with mutexes to let a thread wait until a certain condition is met (for example, "the queue is not empty"). Read-write locks let any number of readers work at the same time, but only one writer. This speeds up workloads that are mostly reading. Monitors and barriers are examples of higher-level constructs that build on these primitives.

A short C example using a POSIX mutex illustrates the pattern:

```
1 #include <pthread.h>
2
3 int counter = 0;
4 pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
5
6 void* increment(void* arg) {
7     for (int i = 0; i < 100000; i++) {
8         pthread_mutex_lock(&lock);
9         counter++;                // critical section
10        pthread_mutex_unlock(&lock);
11    }
12    return NULL;
13 }
```

Listing 1: Protecting a shared counter with a mutex.

Without the mutex, the read–modify–write sequence on `counter` is not atomic, and the final value will be unpredictable (depends on the order of the critical section access) than expected because of lost updates.

2.4 Advantages and Limitations

The main benefit of multi-threading is that it can use multi-core hardware to run tasks in parallel. When properly threaded, CPU-bound workloads like scientific simulation, video encoding, and large matrix operations scale almost linearly with the number of cores.

A second benefit is more abstract. Each thread does its job with regular sequential code, which is much easier to understand and fix than callback-driven asynchronous code, at least for the body of a single thread.

These benefits come with some well-known downsides. **Memory overhead** is the first thing to think about. Each thread needs its own stack, which is usually 1–8 MB on Linux by default. If a server tried to keep ten thousand connections open at the same time with a thread-per-connection model, it would need tens of gigabytes of memory just for stack space. The second thing is the cost of **Context-switching**. The scheduler uses more and more CPU cycles to save and restore state instead of running user code as the number of runnable threads grows beyond the number of cores. This issue, which is sometimes called the C10K problem, led to the rise of event-driven servers in the early 2000s.

Race conditions happen when the correctness of a program depends on the order in which operations happen across threads. If the order is wrong, the program will give wrong results. Deadlocks happen when two or more threads each have a resource that another thread is waiting for. This creates a cycle of dependencies that no one can get out of. The four classic conditions for deadlock that are talked about in books. They are hold and wait, no preemption, circular wait, and mutual exclusion. These are easy to say, but they're surprisingly hard to avoid in big code bases. Some risks that are similar are livelock, where threads stay active but don't make any progress, and priority inversion, where a low-priority thread blocks a high-priority thread and makes it starve forever.

In short, multi-threading provides strong parallelism, but it also uses a lot of memory, takes a lot of time to schedule, and can cause timing-dependent bugs that are hard to reproduce and fix.

3. Event Loop Architecture

3.1 Single-Threaded Event Loop Operation

Event-driven concurrency does the opposite. Instead of creating a new thread for each task, the runtime keeps a single thread that runs an **event loop**. The loop keeps checking for finished I/O operations or other events that are ready. When it finds one, it sends the right handler (callback) to it and then goes back to the loop. There is only one thread, so no two callbacks can run at the same time, and shared data structures don't need locks.

Node.js is the most well-known example of an event loop, but the pattern has been around for a long time. The 1980s X Window System, the GUI message loops of Microsoft Windows and macOS, and high-performance servers like Nginx all use different versions of the same idea. The

unifying idea is that the application never stops; instead, it makes an asynchronous request, registers a callback, and gives control back to the loop, which can handle other ready events while the original request is still in flight.

Node.js's event loop is built on top of the `libuv` library, which hides platform-specific features like Linux's `epoll`, macOS's `kqueue`, and Windows's I/O Completion Ports. Each time the loop runs, it goes through a number of different steps: timers, pending I/O callbacks, idle and prepare handlers, the I/O poll itself, `setImmediate` callbacks, and close callbacks. Promises take care of microtasks that are waiting in line between phases.

3.2 Callbacks, Event Queues, and Non-Blocking Operations

When application code starts an I/O operation, like `fs.readFile` in Node.js, the runtime sends the request to a thread pool or the kernel's asynchronous I/O facility and then gives control back to the event loop right away. The application has a callback function that the runtime will call when the operation is done. When an operation is finished, it goes on an **event queue**. The event loop then takes them off one by one and runs the callbacks that go with them. Every action that the app takes must not block, which is very important.

A callback that does a CPU-intensive calculation in real time, like parsing a five-megabyte JSON document or computing a cryptographic hash, stops the whole event loop because no other callback can run until the current one returns. This is why Node.js apps often let `worker_threads` or external services handle this kind of work.

```
1  const fs = require('fs');
2
3  console.log('Step 1: requesting file');
4
5  fs.readFile('data.txt', 'utf8', (err, contents) => {
6    if (err) return console.error(err);
7    console.log('Step 3: file read finished');
8  });
9
10 console.log('Step 2: event loop continues');
```

Listing 2: Non-blocking file read in Node.js.

The output is 1, 2, 3 rather than 1, 3, 2, because the read operation is queued and the program continues immediately. The callback fires only after the read completes and the event loop reaches the appropriate phase.

3.3 Callbacks, Promises, and `async/await`

The earliest Node.js APIs used the so-called *error-first callback* convention. While this works but deeply nested callbacks like the infamous “callback hell” or “pyramid of doom” quickly become unreadable, and error handling has to be threaded manually through every level.

Promises, standardized in ECMAScript 2015, model an asynchronous value as a first-class object that is initially pending and eventually settles to either a fulfilled or rejected state. Promises can be chained with `.then()` and `.catch()`, which flattens the nesting and centralizes error handling.

async/await, added in ECMAScript 2017, is syntactic sugar over promises that allows asynchronous code to be written in an apparently sequential style. An **async** function implicitly returns a promise, and the **await** keyword suspends execution of the function until the awaited promise settles without blocking the event loop, because the rest of the async function is itself scheduled as a continuation.

```
1 // 1. Callback style
2 fs.readFile('a.txt', 'utf8', (err, a) => {
3   if (err) return done(err);
4   fs.readFile('b.txt', 'utf8', (err, b) => {
5     if (err) return done(err);
6     done(null, a + b);
7   });
8 });
9
10 // 2. Promise style
11 fs.promises.readFile('a.txt', 'utf8')
12   .then(a => fs.promises.readFile('b.txt', 'utf8')
13     .then(b => a + b))
14   .then(result => done(null, result))
15   .catch(done);
16
17 // 3. async / await
18 async function readBoth() {
19   const a = await fs.promises.readFile('a.txt', 'utf8');
20   const b = await fs.promises.readFile('b.txt', 'utf8');
21   return a + b;
22 }
```

Listing 3: The same task in three styles.

3.4 Strengths and Challenges

The event-driven model elegantly sidesteps the worst hazards of multi-threading. Because callbacks run one at a time on a single thread, there are no race conditions on in-process data, no locks to acquire, and no deadlocks. Memory usage is dramatically lower because a Node.js server can comfortably hold tens of thousands of idle connections in a few hundred megabytes of memory as each connection requires only a small descriptor and a few callback closures rather than a multi-megabyte stack.

However, the model introduces its own difficulties. The most significant is that any **blocking operation** on the main thread halts the entire system. A synchronous `JSON.parse` on a large payload, an unbounded loop, or a poorly written regular expression can render a server unresponsive for all clients simultaneously. Programmers must therefore be vigilant about which APIs are blocking, and CPU-bound work must be moved to worker threads or separate processes.

Debugging is also harder. Stack traces in callback-based code are typically truncated at the point where the asynchronous operation was scheduled which causes the loss of the chain of causation that led to the error. Modern runtimes mitigate this with *async stack traces*, but the underlying difficulty that logical control flow no longer matches the call stack remains. The mental model required to reason about “what happens between awaits” is unfamiliar to programmers trained on synchronous code.

Finally, an event-driven server cannot exploit multiple CPU cores by itself. Scaling beyond a single core requires either spawning multiple worker processes (the `cluster` module in Node.js) or using a worker thread pool for parallel CPU work. This is a real limitation for compute-heavy workloads.

4. Asynchronous I/O Systems and Performance Analysis

4.1 Synchronous versus Asynchronous I/O

In a **synchronous I/O** model, when a thread issues a system call such as `read()`, the kernel suspends the thread until the requested data is available. The thread is removed from the run queue. Though it consumes no CPU during the wait, it does occupy memory and a kernel scheduling slot. Synchronous I/O is conceptually simple where the call returns when the work is done but every blocked thread still costs resources.

In an **asynchronous I/O** model, the system call returns immediately even before the operation has finished. The kernel notifies the application later, either by setting a flag, signaling a file descriptor, or invoking a callback. The application is free to do other work in the interim. Linux provides several asynchronous facilities including `epoll` (readiness notification), POSIX AIO, and the more recent `io_uring`, which delivers true completion-based asynchrony with very low overhead.

It is important to note that asynchronous I/O and the event-driven model are not the same thing. A multi-threaded program can use asynchronous I/O if it wishes, and an event-driven runtime such as Node.js may internally use a small thread pool for operations (such as DNS resolution or file I/O on certain platforms) where the underlying kernel does not provide a fully asynchronous interface. The distinction is between *how the application is structured* (one thread per task vs. one event loop) and *how the kernel delivers I/O completions* (blocking system calls vs. readiness or completion notifications).

4.2 I/O-Bound versus CPU-Bound Workloads

The two models behave very differently depending on whether work is dominated by waiting on I/O or by computation.

For **I/O-bound workloads** such as web servers, chat applications, API gateways, database proxies, the event-driven model usually wins decisively. Most of the wall-clock time is spent waiting for the network or disk. The application itself is doing very little CPU work. A single Node.js process can multiplex tens of thousands of mostly idle connections through one event loop with negligible overhead, while a thread-per-connection design would collapse under the memory and context-switching cost long before reaching the same scale.

For **CPU-bound workloads** like video transcoding, scientific simulation, machine learning training, image processing, the multi-threaded model is the natural fit. The work is genuinely parallel, the threads spend almost all of their time computing rather than waiting, and the operating system can keep every core busy. An event loop has nothing to offer here and in fact, its single-threaded execution becomes a hard bottleneck.

Mixed workloads are common in practice. A typical pattern in Node.js is to handle the I/O-heavy

public surface (request parsing, routing, database calls, response serialization) on the event loop while delegating CPU-intensive subtasks to a thread pool or a separate microservice written in a language better suited to numeric work.

4.3 Performance Under High Concurrency

Memory: A pthread on Linux defaults to an 8 MB stack. At ten thousand connections, a thread-per-connection server commits substantial virtual memory and pays for kernel bookkeeping in every TCB or PCB. An event-driven server holds only a small descriptor and a few callback closures per connection, often a few kilobytes total.

Scalability: The event-driven model scales gracefully with the number of concurrent connections, since the cost of an idle connection is essentially zero and the event loop simply has more work to dispatch from. Multi-threaded servers face the C10K problem. As runnable thread counts approach and exceed the number of cores, scheduler overhead grows nonlinearly. Modern designs mitigate this with bounded thread pools and asynchronous I/O. This blurs the boundary between the two models.

Latency and responsiveness: Under steady-state load, the event-driven model often delivers lower and more predictable tail latencies because there are no context switches or lock contention in the hot path. However, a single long-running callback can cause head-of-line blocking that affects every other client at once, whereas in a multi-threaded server one slow request typically only delays the thread handling it.

CPU utilization: Multi-threaded systems can saturate all cores out of the box. Event-driven systems must explicitly use multiple processes or worker threads to do the same; a vanilla single-process Node.js application leaves $N - 1$ cores idle on an N -core machine.

The following table summarizes the comparison.

4.4 Real-World Use Cases

Web servers and API backends: High-concurrency, I/O-heavy workloads strongly favour event-driven runtimes. Nginx (C), Node.js (JavaScript), and asyncio-based Python frameworks routinely handle workloads at which classical thread-per-connection servers such as the original Apache prefork model would fail. This is the canonical success story of the event-driven model.

Real-time applications: Chat servers, multiplayer game lobbies, collaborative editors, and live dashboards involve many long-lived, mostly idle connections and small bursts of activity. The event loop is ideal. Here, the connections cost almost nothing when idle, and the server can broadcast updates to thousands of clients with low latency.

Computational and data-processing systems: Scientific computing, video transcoding, machine learning training, simulation, and large-scale data processing are CPU-bound and benefit from genuine parallelism across cores. Multi-threaded designs in C++, Java, Rust, or Go with goroutines (which are themselves a hybrid of the two models) dominate this space.

Database engines and operating system kernels: These systems mix both styles. The kernel itself is heavily multi-threaded, but it exposes asynchronous I/O facilities to user space.

Aspect	Multi-Threaded Model	Event-Driven Model
Concurrency unit	OS thread (one per task)	Callback/coroutine on a single thread
Memory per task	High (per-thread stack, 1–8 MB)	Low (closure, a few KB)
Context switching	Frequent, kernel-managed	None within the loop
Shared-state safety	Requires mutexes, semaphores, etc.	Naturally safe (no concurrent access)
Multi-core utilization	Native, automatic	Requires worker threads or clustering
CPU-bound work	Excellent fit	Poor fit(blocks the loop)
I/O-bound work	Adequate, costly at high scale	Excellent; scales to 10^4+ connections
Failure modes	Race conditions, deadlocks	Blocking callbacks, callback hell
Debugging	Standard tools, but timing-sensitive bugs are hard to reproduce	Async stack traces required; control flow is non-linear

Table 1: Side-by-side comparison of multi-threaded and event-driven concurrency.

Modern database engines such as ScyllaDB and Redis use event-loop-style architectures pinned to individual cores (a “thread-per-core” design) to combine the scalability of event loops with the parallelism of multi-threading.

Hybrid runtimes: The clean dichotomy between the two models is increasingly blurred. Go’s goroutines look like threads to the programmer but are multiplexed by the runtime onto a small pool of OS threads using a non-blocking scheduler. The Java Virtual Machine has added virtual threads (Project Loom) along the same lines. These hybrids preserve the readability of synchronous code while keeping memory usage and context-switching costs close to those of an event loop.

5. Conclusion

Thread-based concurrency and event-driven concurrency are not competing answers to the same question. They are answers to different questions. Multi-threading exists primarily to exploit parallel hardware, and it does so well at the cost of memory, scheduling overhead, and a class of synchronization bugs whose pathologies have shaped decades of operating-systems research. The event loop exists primarily to scale to enormous numbers of mostly idle I/O-bound tasks, and it does so well by sacrificing parallelism within a single process and forcing the programmer to decompose work into non-blocking callbacks.

For a systems engineer, the practical conclusion is that the choice of model should follow the workload. I/O-bound, high-fan-out services such as web frontends, real-time messaging, and API gateways naturally fit the event loop. CPU-bound workloads such as scientific computing and media processing naturally fit threads. Most production systems blend the two. An event-driven outer layer for connection handling, with thread pools or separate processes for parallel CPU work, and an operating system underneath that uses both internally. Understanding the

strengths, costs, and failure modes of each model is therefore not a matter of picking a winner, but of selecting and combining tools intelligently for the problem at hand.